

Ask anything

Yes, you can write a comprehensive, publication-quality interactive textbook using Jupyter Book, and it is widely used for data science and STEM courses (such as UC Berkeley's famous Data 8 textbook). Jupyter Book +1

What Jupyter Book Does Well

- **Blends Prose and Code:** Write using Jupyter notebooks (`.ipynb`) or an advanced flavor of Markdown called MyST Markdown.
- **Executes Code on Build:** It runs your code snippets during the build process so that charts, tables, and numerical outputs update and appear inline automatically.
- **Academic Features:** Supports advanced math equations, citations, cross-references ("see Figure 2"), and labeled theorem blocks.
- **Dual Outputs:** Compiles your single source files into both dynamic online HTML websites and printable PDF outputs. ACS Publications +3

Limitations of Jupyter Book

- **No Hosted Dashboard:** It is an open-source static site generator, not a managed platform. You must configure hosting yourself (e.g., using GitHub Pages) and manage your own deployment workflow. YouTube · Open Geospatia...
- **Complex PDF/Print Toolchain:** Generating professional, print-ready PDFs requires installing and debugging an external LaTeX engine, which can be finicky. Docsio
- **Stiff Learning Curve for MyST:** While simple Markdown is easy, using advanced features like grids, customized admonitions, or specific academic directives requires learning MyST syntax.

AI Mode All Short videos Forums Videos More

If you want to move forward, tell me:

- What **subject or programming language** is your textbook about?
- Do you prefer to write in **pure Markdown** or **Jupyter notebooks**?

I can share a **basic setup guide** or **recommend alternative tools**.

YouTube · Open Geospatial Solutions

Creating an open-source book with Jupyter Book and Jupytext

Creating an open-source book with Jupyter Book and Jupytext hello in this video I'm going to show you how to create uh an open sou...



GitHub Pages documentation

Books with Jupyter - GitHub Pages

Jupyter Books lets you build an online book using a collection of Jupyter Notebooks and Markdown files. Its output is similar to t...

Docsio

Jupyter Book Review 2026: Docs From Notebooks - Docsio

MyST Markdown and notebooks Content comes from two sources. The first is Jupyter notebooks (. ipynb), the familiar mix of code c...

Show all

The book would be more along the lines of a firmware and hardware engineering domain. Does that make sense? ...

snippets for microcontrollers. If you use a Python-based hardware framework (like MicroPython, CircuitPython, or Verilog/VHDL analysis tools), Jupyter Book can **execute the code during the build** to auto-generate register maps, memory tables, or logic wave graphs.

- **Excellent for Tool Guides:** The mix of structured text, terminal command blocks, and step-by-step formatting is ideal for explaining how to use electronic tools (e.g., logic analyzers, oscilloscopes, or flash programmers).
- **Version Control for Hardware Docs:** Writing your book in text-based Markdown allows you to track your documentation in Git right alongside your actual hardware design files (schematics, PCB layouts, and source code).

Domain-Specific Limitations & Solutions

While Jupyter Book is a great fit, you will run into a few specific hurdles regarding hardware visuals:

- **Static Images vs. Native Formats:** You cannot easily "embed" native hardware files (like Altium, KiCad, or Eagle schematics) so they can be panned or zoomed dynamically. You will have to export your schematics as high-resolution **SVG or PNG files** and link them as standard images.
- **Lack of Visual WYSIWYG Editor:** Since you are describing hardware tools, you'll likely need lots of callouts, arrows, and diagrams. Because Jupyter Book relies on code/Markdown, you cannot drag-and-drop text boxes over an image of an oscilloscope. You must edit the image in an external tool (like Inkscape or Draw.io) and then import it.
- **Diagram-as-Code Limitations:** Jupyter Book natively supports **Mermaid.js** for making text-based block diagrams or flowcharts. However, it does *not* natively draw electrical circuits or timing diagrams from code.

Tool	Best Used For
Jupyter Book	Best if you want to include **executable scripts**, interactive register calculators, or code-generated timing diagrams.
MkDocs (with Material theme)	Better if you want a **faster, simpler text-and-image guide** without a Python code execution engine running in the background.
Sphinx	The industry standard for heavy engineering documentation, though it uses ReStructuredText (RST) which has a steeper learning curve than Markdown.

To help you map out the best workflow, let me know:

- Do you plan to include **interactive elements** (like a Python-based bitmask/register calculator or live wave diagrams)?
- What is your target output—a **publicly hosted website** or a **downloadable PDF**?

So, it works fine with things like PNG, what about JPG? There's a lot of JPG images that I have

Yes, Jupyter Book works **perfectly fine with JPG images**. You can use **JPG, PNG, SVG, and even animated GIFs** across your pages without any issues.

Because Jupyter Book compiles into standard HTML (for websites) and PDF, it

```
![Using the logic analyzer to capture SPI data](../images/logi
```

Use code with caution.

- **MyST Markdown Figure (Best for Textbooks):**
  This method allows you to number your figures, add rich captions, cross-reference them in your text (e.g., *"As seen in Figure 1"*), and control the width.

```
markdown
```

Use code with caution.

```
```{figure} ../images/oscilloscope_probe.jpg

width: 80%
name: oscilloscope-probe-fig

Connecting the ground clip of the probe to the nearest chassis ground.
```
```

```
### A Quick Warning for Hardware Textbooks
While JPG is excellent for **real-world photographs** (like a pict

For **schematics, circuit diagrams, and code screenshots**, try to
```

both?

Yes, **you can absolutely have both.** One of the greatest strengths of Jupyter Book is its ability to build **a beautiful, searchable website AND a downloadable, print-ready PDF** from the exact same source files.

When you publish your book, the online version will include a **built-in download button** right at the top of the page, allowing your readers to download the entire textbook as a single PDF.

How the Dual Setup Works

1. **The HTML Website:** Jupyter Book compiles your Markdown and Jupyter notebooks into static HTML files. You can host this for free on platforms like **GitHub Pages**. It includes a left-hand navigation sidebar, search functionality, and full support for your JPG photos.
2. **The PDF Version:** Jupyter Book takes those same source files and converts them into a PDF. You have two ways to generate this PDF:
    1. **Via Web Browser (Easiest):** It uses a headless web browser to "print" your website pages into a PDF file. This preserves your CSS styling, web fonts, and layouts exactly as they look online.
    2. **Via LaTeX (Advanced):** It converts your book into a standard LaTeX document and compiles it. This is harder to configure, but it yields a highly professional, traditional academic book layout.

A Crucial Limitation for Hardware Books (HTML vs. PDF)

While you get both outputs, you must design your hardware textbook with the



rates or bitmasks, it will be fully interactive online but will just display as a static code snippet or empty block in the PDF.

The Recommendation

Most engineering authors use the **web-browser PDF engine** (`pyppeteer`) because it handles image scaling and web layouts much more gracefully than LaTeX, especially when dealing with dozens of JPG tool photographs.

If you'd like, I can show you the **exact folder structure** and configuration file (`_config.yml`) you need to set up a project that outputs both a website and a PDF. How does that sound?

OK, how do i get started with Jupyter book?

To get started with Jupyter Book, you need to **install it on your computer using Python, create a template project, and use the command line to build your book**.

Here is the step-by-step guide to setting up a project structured for both **HTML** and **PDF** outputs.

Step 1: Install Jupyter Book

Open your terminal (Mac/Linux) or Command Prompt/PowerShell (Windows) and install Jupyter Book using `pip`. It is also highly recommended to install

Step 2: Create a Template Book

Jupyter Book can automatically generate a sample project structure for you. Run the following command to create a new book project named `my_hardware_book`:

bash

```
jb create my_hardware_book
```

Use code with caution.

Step 3: Understand the Folder Structure

Navigate into your new directory. For a hardware and firmware textbook with lots of JPG photos, you should arrange your folders to look like this:

text

```
my_hardware_book/
├── _config.yml          # Global settings (Title, Author, PDF togg
├── _toc.yml             # Table of Contents (Defines chapter order
├── intro.md             # The homepage / introduction of your book
├── images/              # Folder where you store your JPG/PNG file
│   ├── multimeter.jpg
│   └── schematic1.png
├── chapter1/            # Organized chapters
│   ├── tools.md
│   └── firmware.ipynb   # Can be Markdown (.md) or Notebook (.ipyn
└── chapter2/
```

```yaml
title: Firmware & Hardware Engineering Guide
author: Your Name
logo: images/multimeter.jpg  # Optional logo for sidebar

# Enable repository buttons (handy if hosting on GitHub)
html:
  use_repository_button: true
  use_issues_button: true

# Enable download buttons for PDF
sphinx:
  config:
    html_theme_options:
      download_buttons: true
```

Use code with caution.

Step 5: Build Your Book

Whenever you write new content or add images, you must "build" the book to see your changes.

- **To build the HTML website:**

```bash
jb build my_hardware_book/
```

Use code with caution.



This will compile your book into a single, cohesive PDF file located in the `_build/pdf/` folder.

Would you like me to show you how to write the **Table of Contents (`_toc.yml`)** file to organize your chapters, or would you prefer guidance on **hosting the website for free on GitHub Pages**?